

1. DOM Elements – What They Are & Why They Matter

Element	Purpose	Notes / Importance
audio	Core <code><audio></code> object	Handles all audio playback. <code>preload="metadata"</code> ensures duration is ready, so seekbar & timer work immediately.
playBtn, prevBtn, nextBtn	Player control buttons	Clicked by user → trigger play/pause or navigation functions.
songInfoScroll, songInfoContainer	Display scrolling song title	Shows current song name; scrolling animation applied if name exceeds container width.
searchBar	Input field to search songs	Filters <code>songs</code> array dynamically; allows quick access to any song.
playlistUL	<code></code> element to display all songs	Clicks on <code></code> items play corresponding song; uses event delegation for efficiency.
seekbar, seekCircle	Visual playback progress bar	Circle shows current playback position; can drag to change time.
songTimer	Displays <code>current / total</code> song time	Updates automatically with audio playback.
volumeInput	Slider to control volume	Maps input from 0–100 → <code>audio.volume</code> 0–1.
hamburger, closeBtn, leftSidebar	Sidebar navigation	Toggle playlist/sidebar visibility.

2. Core Functions – How They Work & Why

2.1 getSongs(folder = "Saiyaara")

- **Purpose:** Fetches songs for a specific album folder from `songs.json`.
- **Flow:**
 - Checks **cachedPlaylists** → returns immediately if already loaded (performance).
 - Uses `fetch()` to get `songs.json`.
 - Converts filenames to full paths: `SONGS/folder/filename`.
 - Stores them in cache as objects `{ href, name }`.
 - Returns array of songs.

- **JS concepts:** Async/await, fetch API, error handling, caching.
- **Why needed:** Makes album selection dynamic; avoids reloading same data repeatedly.
- **Note:** Always returns an array; empty if folder not found or fetch fails.

2.2 updateSongName(name)

- **Purpose:** Scrolls song title horizontally if it's wider than the container.
- **Flow:**
 - Set `songInfoScroll.textContent = name`.
 - Reset animation to none.
 - Compare `scrollWidth` vs container width.
 - If text overflows → set CSS animation duration proportional to text width.
- **JS concepts:** DOM manipulation, element measurements (`scrollWidth`, `clientWidth`), CSS animation via JS.
- **Why needed:** Ensures long song names are readable without truncation.

2.3 formatTime(sec)

- Converts seconds to `"mm:ss"` format for display.
- Used in **seekbar timer** and song info.
- Example: `formatTime(75)` → `"1:15"`
- Simple utility function; helps user understand song progress at a glance.

2.4 updateSeekbar

- Updates seekbar circle according to `audio.currentTime`.
- Avoids updating while user is dragging (`isDragging`).
- Calls `updateTimer()` to keep time text synced.
- **JS concepts:** DOM manipulation, arithmetic, conditional logic.

2.5 updateTimer

- Updates `songTimer` with `current / total` time.
- Uses `formatTime()` internally.
- Provides real-time feedback of playback.

2.6 playSong(index)

- Sets `audio.src` to `filteredSongs[index].href`.
- Plays song asynchronously using `audio.play()`.
- Updates play button icon and scrolling song name.
- Handles autoplay restrictions via try/catch.
- Updates `currentIndex` and `isPlaying`.

2.7 togglePlay

- Pauses or plays audio depending on `isPlaying`.
- Updates play button icon accordingly.

2.8 nextSong / prevSong

- Moves to next or previous song in `filteredSongs`.
- Uses modulo arithmetic to wrap around playlist.
- Calls `playSong(currentIndex)`.

2.9 renderPlaylist(list)

- Updates `playlistUL` with `<li>` elements for each song.
- Updates `filteredSongs`.
- Handles empty list with `No results found`.
- Uses `.map()` and template literals for dynamic HTML.

2.10 Playlist Click Handler

- Delegated event listener on `playlistUL`.
- Detects clicked `<li>` using `e.target.closest('li')`.
- Ignores clicks on empty areas or "no results".
- Finds index via `Array.from(playlistUL.children).indexOf(li)` and calls `playSong(index)`.

2.11 Search Bar Input Handler

- Filters songs on each keystroke (`input` event).
- Updates `filteredSongs` and resets `currentIndex`.
- Calls `renderPlaylist(filtered)` to update UI.

2.12 seekTo(e)

- Calculates clicked or dragged position on seekbar.
- Sets `audio.currentTime` based on percentage.
- Moves `seekCircle` visually and updates timer.

2.13 Volume Control

- Listens for changes on volume slider.
- Maps 0-100 input → 0-1 audio volume.

2.14 Album Card Click Handler

- Fetches songs from album folder using `getSongs(folder)`.
- Renders playlist and plays first song.
- Stops event propagation to prevent unintended clicks.

2.15 Hamburger / Sidebar Handlers

- Toggles sidebar using `.show` CSS class.
- `hamburger` → show/hide, `closeBtn` → hide.

2.16 Audio Event Listeners

- `timeupdate` → `updateSeekBar()`.
- `ended` → `nextSong()`.
- Keeps UI in sync with playback automatically.

2.17 Button Event Listeners

- `playBtn` → `togglePlay()`.
- `nextBtn` → `nextSong()`.
- `prevBtn` → `prevSong()`.

3. Key State Variables

Variable	Purpose
songs	All songs loaded for current album
filteredSongs	Songs filtered via search
currentIndex	Index of currently playing song
isPlaying	True if audio is playing
isDragging	True if user is dragging seekbar
cachedPlaylists	Caches loaded folders to avoid repeated fetches

4. Notes / Pitfalls

- Autoplay may fail on some browsers → handled in `playSong()`.
- Ensure CSS positioning for `seekCircle` (absolute/relative).
- Event delegation avoids adding many listeners.
- Large playlists → consider debouncing search input for performance.
- Circular playlist logic with `%` prevents index errors.
- All functions are written to be **browser-compatible** for modern browsers.

This revision guide can now be exported as a **PDF** for studying, with full explanation of all JS logic, variables, and events.